
camera module - Billard@ISEM

Release 0.1

Mathis Wolter

Mar 12, 2026

CONTENTS:

1	Camera Module for Billard@ISEM	3
1.1	Hardware setup	3
1.2	Installation	3
1.3	Calibration	4
1.3.1	Lens correction	4
1.3.2	Image cropping	5
1.4	Getting images	5
2	API	7
2.1	Camera	7
2.1.1	Camera	7
2.2	BallDetector	13
2.2.1	BallDetector	13
2.3	VideoCapture	14
2.3.1	VideoCapture	14
2.4	vpi_camera_calibration_capture	15
3	To Do	17
	Python Module Index	19
	Index	21

The Camera Module is a part of the [Billard@ISEM](#) system, ultimately aiming at teaching students relevant mechanical design principles as well as provide a research environment for agile product development methods. The Camera Module provides a livestream and has the ability to detect the ball position and type using YOLO models. It runs on the Nvidia Jetson platform.

CAMERA MODULE FOR BILLARD@ISEM

Implementation of a computer vision system for the [Billard@ISEM](#) system.

This module provides functionality for image calibration and an API for livestreaming, inferring billiard ball coordinates/numbers and much more.

1.1 Hardware setup

- Currently using a Nvidia Jetson Orin Nano with a Raspberry HQ Camera.
 - Any other Nvidia Jetson should do, as we rely on their VPI library for image processing. Older commits (before the branch `jetson-port`) work on any computer by using CPU resources instead of hardware accelerations, which is mostly slower.
- The current lens correction setup is only for lenses where a polynomial lens correction is applicable (most normal lenses). If your camera setup uses a lens, for which a fisheye model is more applicable, you would have to modify some parts of the source code.

1.2 Installation

The installation process for this module is quite complex, since it relies on a lot of optimisation steps. The installation can take multiple hours, depending on your hardware.

1. Clone this repository: `git clone --recurse-submodules https://github.com/ISEM-TUHH/billiard-camera-module.git`
 - Downloading into the `/home/<user>` directory simplifies the next steps.
2. Edit `config/config.json` to have the correct IP-addresses of your other modules.
3. Move into the installation directory: `cd billard-camera-module/install`
4. Edit `billiard-camera-server.service`: correct the user from `isem` to whatever the user name on your system is. Also correct the paths to the executable/directory, if you did not clone into the home directory.
5. To start the main installation process, run `sudo chmod +x ./install.sh && sudo ./install.sh`
 - This will make the install script executable and run it.
 - The script will create a virtual python environment (called `.venv`) and install the dependencies
 - This includes building `opencv-python` from source to be compatible with cameras connected via CSI. From experience, this takes a long time.
 - A new `systemd` service (in `/etc/systemd/system/`) called `billard-camera-module.service` will be created, enabled but not started.

- This is the main service that should run when the system is used. Starting it is shown later on in this document.
- A root cronjob to reboot the system every night at 4:00am will be created. From our experience, this solves a lot of problems.
- If you do not want some of these steps, you can comment them out in the `install.sh` file.
- 6. From the root directory of the repo, run `./launch.sh` to test the system. It should start the camera with the configuration from `config/config.json`, available under the port described in that file.
 - The video shown in the browser is very likely weird, as you still need to calibrate the camera.
 - If you have a camera with manual focus and already mounted your camera in the desired position, you can use the livestream to set the focus.

1.3 Calibration

The calibration processes are supported by physical objects, mostly certain visual patterns. See [resources/](#) for the exact objects we used.

1.3.1 Lens correction

Unless your camera comes pre corrected, you'll need to correct for lens distortion. If you want more details on lens distortion, read up for example [here](#).

1. Have a printout of a calibration checkerboard pattern available.
 - From our experience, you'll get the best results if you do this after mounting and focusing the camera in the final position (see 6. from Installation).
 - Checkerboard patterns can be generated as .pdf [here](#) (third party). This is only tested for "Target Type" "Checkerboard".
 - The checkerboard size should be sufficient to be well visible in the camera. We used an A0 print at approx. 3m distance.
 - Place the printed pattern on some rigid surface. There should be no curves or creases.
2. Navigate to the root of the repo
3. Activate the virtual environment: `source .venv/bin/activate`
4. Capture calibration images using `python vpi_camera_calibration_capture.py`
 - If your camera has a different framerate to 21, edit this in the file before.
 - This script captures a total of 15 images, one every 8 seconds
 - Between each capture, move the checkerboard around to have maximum coverage of the image. Also rotate the pattern.
 - There now should be 15 new images in `config/calibration_images`
 - You can repeat this as often as you want until you think your images are good enough.
5. Run the calibration script: `python vpi_camera_calibration_analysis_polynomial.py`
 - If your checkerboard pattern has a different size than 15x21 rows/columns, change the `CHECKERBOARD` variable in the script. It should contain the number of borders between rows/columns.
 - This script takes all the .png images in directory `config/calibration_images` and uses them to calculate the best fitting polynomial lens correction parameters

- Results are saved into `config/distortion_correction.json` and an exemplary corrected image in `config/calibrated_images`.

6. The next time the camera software starts, it will use the saved lens correction parameters.

1.3.2 Image cropping

We want to crop the image to the corners of the billard table, so that the camera infers correct coordinates. If you also use the [Beamer Module](#), you should calibrate them onto the same corners. This must be done with the Camera Module software running, as it is done dynamically.

1. Create a set of markers to mark the corners.
 - Print 4 ArUcO markers of the dictionary/family 4x4, the IDs 1, 2, 3 and 4. We use an edge length of approx. 76mm.
 - When printing and cutting them out, be aware that a small white border around the marker is essential. Also keep track of which marker is which.
 - Create a fixture of some kind to reproducibly place the markers in the corners of the table. A lasercut template is available in the `resources/` directory.
 - With our solution: by pushing the fixture into the corner, place the corresponding marker onto the fixture such that the corner is directly beneath the edges of the billard table.
2. Open the website of the module
3. Open the link `/v1/calibrate` in a new tab (background)
4. Now, the livestream video should be zoomed out and looking for the four markers.
 - As soon as all four are found, the image crops onto them.
 - Found markers are drawn with a (thin) green line around them
 - If your camera has problems finding them, we found the following debugging steps to be helpful:
 - With a manual focus camera, check all corners are in focus
 - Change the lighting
 - Consider printing larger markers
5. If they are zoomed in correctly, you can now click on `v1/savetransformation` from the modules homepage to save it.
 - In the future, this configuration will be used.
6. You are done and can now remove the markers. This calibration only needs to be repeated when you change something.

Alignment of the first ArUco marker in the top-left corner

1.4 Getting images

Every now and then, you might want to download/extract all files that can be used in training. Use `zip images.zip images/**/*` to zip all images in subfolders of the `images` directory.

<i>Camera</i>	
<i>BallDetector</i>	
<i>VideoCapture</i>	
<i>vpi_camera_calibration_capture</i>	Capture raw images from the connected CSI Camera

2.1 Camera

Classes

<i>Camera</i> ([config, test_config, template_folder])	Camera module for the Billard@ISEM system.
--	--

2.1.1 Camera

```
class Camera.Camera(config='config/config.json', test_config='config/test_config.json',  
                    template_folder='templates')
```

Bases: Module

Camera module for the [Billard@ISEM](#) system.

```
__init__(config='config/config.json', test_config='config/test_config.json', template_folder='templates')
```

Setup the instance.

This defines the API interface (see `api_dict` in source).

Methods

<i>__init__</i>	Initialize self.
-----------------	------------------

Attributes

<code>aruco_dict</code>	cv2 ArUco dictionary of 4x4 family
<code>counterPictures</code>	the number of images saved with incremental names
<code>detector</code>	cv2 ArUco detector
<code>h</code>	height and width of the picamera image
<code>lastPing</code>	latest ping from a liveline call (timestamp)
<code>lastPositions</code>	last measured coordinates
<code>lastVideoFrame</code>	the latest frame that was generated
<code>latestFrameTime</code>	timestamp of the latest generated frame
<code>ph</code>	9ft pool table measurements 223x111.5cm
<code>pw</code>	9ft pool table measurements 223x111.5cm
<code>recalibrate</code>	state to track if there is a call to recalibrate the image to the ArUco markers
<code>videoStreaming</code>	tracks if there is a current videostream
<code>w</code>	height and width of the picamera image
<code>zoomout</code>	state to track if there is a call to zoom out the camera view (remove calibration)
<code>worker_uuid</code>	this tracks which uuid (client connection) actually generates the frame, all other connections only listen and send to cached frames.

aruco_dict = < cv2.aruco.Dictionary 0xffffd443f0b0>

cv2 ArUco dictionary of 4x4 family

beamer_calibration(frame)

Looks for the beamer tags in the frame.

If all four tags are found, sends the coordinates to the beamer module.

Returns the annotated frame, annotates found markers.

cache_image()

Caches the current frame

This is used for later on saving the current image for training purposes if the coordinates where corrected.

Returns

Message that the image was cached

Return type

str

counterPictures = 0

the number of images saved with incremental names

detector = < cv2.aruco.ArucoDetector 0xfffff073f1fd0>

cv2 ArUco detector

do_calibrate()

Raise the flags for the calibration procedure

Raises Camera.recalibrate and Camera.zoomout, which cause the Camera.gen method to zoomout and look for new ArUco markers.

Returns

rendered *calibration_tutorial.html* to guide the user through the calibration.

Return type

Response

do_savepic()

Takes the last taken image from the buffer (self.lastVideoFrame) and writes them to ./images/ as .png If json data has the tag “action”: “save-labels”, look for coordinates in the data and save them in the YOLO label format.

Returns

jsonified name of the image.

do_zoomout()

Raise the flag to zoomout

This causes the camera stream to zoomout.

Returns

“tiptop”

Return type

str

force_restart()

This function kills the process (stops the server).

This should restart the server, as it is listed in systemctl with restart=always

Returns

“Restarting the server.” (this should never reach the client, as the server terminates before)

Return type

str

gen(*once=False, own_uuid=None*)

Generate an image (stream) and yield on every generation. Prevent double generation from different clients by writing to self.lastVideoFrame and returning that instead of generating an entire new image.

THIS RETURNS A GENERATOR OBJECT (due to yield, even if they are not reached in the structure). To actually execute this outside of video_feed, put it in a “for i in self.gen(...): continue”. Only this actually calls the functions :)

Parameters

once (*optional bool*) – take just one image (run through the entire method once)

get_coords(*image=None*)

Detect billiard balls in the current frame

The current frame (gathered using *Camera.get_image_internal*) and passes it to the *Camera.ballDetector.detect*.

Returns

flask response containing json of the coordinates in real dimensions.

Return type

Response

get_game_image()

Get a GameImage of the current ball positions

This is a debug tool to quickly check if corrections are working correctly. The GameImage used here is simplified compared to the implementation used in the game module (see <https://github.com/ISEM-TUHH/billard-game-module/blob/main/Game/GameImage.py>).

Returns

flask response of mimetype *image/jpg*

Return type

Response

get_image()

Get the current frame

Returns

flask response of mimetype *image/jpg*

Return type

Response

get_image_internal()

Retrieve the current image from the camera

Either uses the latest frame from the livestream or generates a single new frame.

Returns

cv2 image

Return type

np.ndarray

h = 3040

height and width of the picamera image

index()

Renders and returns the index.html website

lastPing = 0

latest ping from a liveline call (timestamp)

lastPositions = None

last measured coordinates

lastVideoFrame = 0

the latest frame that was generated

latestFrameTime = 0

timestamp of the latest generated frame

liveline()

Updates the Camera.lastPing to the current time

This is an old mechanic to notice disconnects. This is not needed anymore since commit b9696d4e92e815e172aa00cb59d3a2ef3cbe6717 added closing the camera generation/stream on client disconnect. It still exists as some legacy websites hit up this endpoint.

Returns

“tiptop”

Return type

str

load_distortion_correction()

Load the distortion correction from storage.

Load the Camera.[mtx, dist, rvecs, tvecs] from *config/distortion_correction.json*. These should be calculated by *vpi_camera_calibration_analysis_polynomial.py* using *cv2.calibrateCamera*.

load_matrix()

Load the transformation matrix from transformation-matrix.json

If images are being generated and not zoomed out, this transformation is applied to them.

Returns

Message containing the matrix

Return type

str

ph = 1115

9ft pool table measurements 223x111.5cm

pw = 2230

9ft pool table measurements 223x111.5cm

reassign_worker_gen()

This function looks at all registered uuid (client connection) and selects the next connection that should be the worker. The newest one is chosen.

Returns

the worker thread uuid

Return type

uuid.UUID

recalibrate = False

state to track if there is a call to recalibrate the image to the ArUco markers

save_matrix()

Save the current transformation

The current matrix Camera.M gets written to *config/transformation-matrix.json*.

Returns

Message containing the matrix

Return type

str

send_quick_inference(frame)

Detect balls and send the coordinates in simple format to the Beamer Module.

Simple format is defined as {"points": [{"x": 123, "y": 123}, {...}, ...]}

start_beamer_calibration()

When this is hit (http GET), starts looking for the beamer markers in Camera.gen (must be running)

Camera.gen calls Camera.beamer_calibration on every loop, see that method for details. This endpoint gets hit up by the beamer module.

stop_generation()

Forcefully stop the camera stream

Returns

"Deactivated stream"

Return type

str

stop_my_stream(uuid)

Stop a stream thread by removing the uuid.

Also reassigns the worker uuid to the most current one.

Parameters

uuid (*uuid.UUID*) – the uuid to remove

Returns

the passed uuid

Return type

uuid.UUID

toggle_quick()

Toggles Camera.do_quick_inference

This causes Camera.gen to always pass the image to *Camera.quickDetector.detect* and send the detections directly to the beamer module.

Returns

Message whether the quick inference has been enabled or disabled

Return type

str

videoStreaming = False

tracks if there is a current videostream

video_feed()

Stream the camera to the client

To prevent generating multiple frames for multiple clients, each stream is assigned a uuid. This allows the system to track that only one thread generates images, with all the others only receiving cached frames. When closing the Response, the uuid is removed from the set of active streams (*Camera.stop_my_stream*)

Returns

flask response of mimetype *multipart/x-mixed-replace; boundary=frame*

Return type

Response

w = 4032

height and width of the picamera image

white_balance(vpi_image, stats_image)

Performs a color correction based on the grey world algorithm on a provided image using the VPI API.

Performance benchmark on a 1115x2230 BGR8 image (Jetson Orin Nano Super): without mean ~1ms, with mean ~65ms on a 4032x3040 image. Discussions can be held on the frequency of resetting the white balance. The default is every 105 frames (5s).

Parameters

- **vpi_image** (*vpi.Image*) – image to get corrected. Should be BGRA8 format, BGR8 should also work.
- **stats_image** (*vpi.Image*) – image on which the scaling should be calculated. This separation is useful if only a small portion of this image actually gets used further on, but the entire image is needed for white balance

Returns

the color corrected image

Return type
vpi.Image

worker_uuid

this tracks which uuid (client connection) actually generates the frame, all other connections only listen and send to cached frames. This can dynamically change through Camera.reassign_worker_gen. If None the stream will end.

zoomout = False

state to track if there is a call to zoom out the camera view (remove calibration)

2.2 BallDetector

Classes

<code>BallDetector</code> ([mode, debug, correction])	Class implementing advanced detection algorithms and plausability checks for a given image and a selected gamemode
---	--

2.2.1 BallDetector

class BallDetector.**BallDetector**(mode='8pool-simple', debug=True, correction=None)

Bases: object

Class implementing advanced detection algorithms and plausability checks for a given image and a selected gamemode

Parameters

- **mode** (*optional string*) – select a gamemode (and detection accuracy/level). Available are currently 8pool-simple and 8pool-detail (simple only differentiating half and full balls, detail every single balls number).
- **debug** – decide if debug statements (like timings etc) should be printed
- **debug** – optional bool

__init__(mode='8pool-simple', debug=True, correction=None)

Methods

<code>__init__</code>	Initialize self.
-----------------------	------------------

Attributes

detect(*img*, *plausability=True*, *plot=True*, *img_name=False*)

Detect pool balls on an image based on the mode selected on init

Parameters

- **img** (*cv2 image object*) – cv2 image object (or link to an image readable by YOLO)
- **plausability** (*bool*) – if True, the result can only have each class once. If False, the same ball class can be detected multiple times
- **plot** (*bool*) – if True, a plot with the images and confidence matrix is generated for debugging
- **img_name** (*str, bool*) – If the img will get saved because of debug decisions, it will get the current datetime as a default name. For batch processing this can be problematic, so the name can also be directly set.

Returns

dict<list of dict(name, x, y, conf), dict<gamemode(str)> of coordinates of detected balls, and used gamemode

saveDebugImage(*folder*)

If the folder is not False, this will save the BallDetector.img to images/[folder]/[self.img_name or timestamp].png

toRealDim(*results, dimensionsTable*)

Returns results[“results”] but with x and y as floats

verify(*img, results*)

Overlay an image with its results and save it. IS BROKEN RIGHT NOW

Parameters

- **image** – cv2 image on which the results where inferred using BallDetector.detect
- **results** – output of BallDetector.detect

2.3 VideoCapture

Classes

<i>VideoCapture</i> (**kwargs)

A bufferless video streaming class.

2.3.1 VideoCapture

class VideoCapture.**VideoCapture**(**kwargs)

Bases: object

A bufferless video streaming class.

Wraps Nvidia’s jetcam CSICamera object for a CSI Camera connected to the Jetson Orin Nano.

running_instances

number of instances reading from this object in parallel

Type

int

__init__(kwargs)**

Create a VideoCapture object

Parameters

****kwargs** – Arguments passed to jetcam.csi_camera.CSICamera

Methods**__init__**

Initialize self.

read()

Get the most current frame

Returns

cv2 image

Return type

np.ndarray

start_stream()

Start the parallel thread reading the camera to empty the buffer queue

If the thread is already running, it just increases the number of running_instances (tracker, no other implications)

Returns

None

Return type

None

stop_stream(force=False)

Stops the stream

This halts the reading thread and waits for 0.1 seconds.

Parameters

force (*bool*, *optional*) – soft stop would only decrease the number of running_instances by one (if that is zero, the thread terminates) or just terminate the thread.

Returns

None

Return type

None

2.4 vpi_camera_calibration_capture

Capture raw images from the connected CSI Camera

Captures 15 images (one every 8s) and saves them to *config/calibration_images*. If you use a different camera, you have to make some adjustments to the camera settings.

CHAPTER
THREE

TO DO

PYTHON MODULE INDEX

b

BallDetector, [13](#)

c

Camera, [7](#)

v

VideoCapture, [14](#)

vpi_camera_calibration_capture, [15](#)

Symbols

`__init__()` (*BallDetector.BallDetector method*), 13
`__init__()` (*Camera.Camera method*), 7
`__init__()` (*VideoCapture.VideoCapture method*), 15

A

`aruco_dict` (*Camera.Camera attribute*), 8

B

`BallDetector`
 module, 13
`BallDetector` (*class in BallDetector*), 13
`beamer_calibration()` (*Camera.Camera method*), 8

C

`cache_image()` (*Camera.Camera method*), 8
`Camera`
 module, 7
`Camera` (*class in Camera*), 7
`counterPictures` (*Camera.Camera attribute*), 8

D

`detect()` (*BallDetector.BallDetector method*), 13
`detector` (*Camera.Camera attribute*), 8
`do_calibrate()` (*Camera.Camera method*), 8
`do_savepic()` (*Camera.Camera method*), 9
`do_zoomout()` (*Camera.Camera method*), 9

F

`force_restart()` (*Camera.Camera method*), 9

G

`gen()` (*Camera.Camera method*), 9
`get_coords()` (*Camera.Camera method*), 9
`get_game_image()` (*Camera.Camera method*), 9
`get_image()` (*Camera.Camera method*), 10
`get_image_internal()` (*Camera.Camera method*), 10

H

`h` (*Camera.Camera attribute*), 10

I

`index()` (*Camera.Camera method*), 10

L

`lastPing` (*Camera.Camera attribute*), 10
`lastPositions` (*Camera.Camera attribute*), 10
`lastVideoFrame` (*Camera.Camera attribute*), 10
`latestFrameTime` (*Camera.Camera attribute*), 10
`liveline()` (*Camera.Camera method*), 10
`load_distortion_correction()` (*Camera.Camera method*), 10
`load_matrix()` (*Camera.Camera method*), 11

M

`module`
 BallDetector, 13
 Camera, 7
 VideoCapture, 14
 vpi_camera_calibration_capture, 15

P

`ph` (*Camera.Camera attribute*), 11
`pw` (*Camera.Camera attribute*), 11

R

`read()` (*VideoCapture.VideoCapture method*), 15
`reassign_worker_gen()` (*Camera.Camera method*), 11
`recalibrate` (*Camera.Camera attribute*), 11
`running_instances` (*VideoCapture.VideoCapture attribute*), 14

S

`save_matrix()` (*Camera.Camera method*), 11
`saveDebugImage()` (*BallDetector.BallDetector method*), 14
`send_quick_inference()` (*Camera.Camera method*), 11
`start_beamer_calibration()` (*Camera.Camera method*), 11
`start_stream()` (*VideoCapture.VideoCapture method*), 15

`stop_generation()` (*Camera.Camera method*), [11](#)
`stop_my_stream()` (*Camera.Camera method*), [11](#)
`stop_stream()` (*VideoCapture.VideoCapture method*),
[15](#)

T

`toggle_quick()` (*Camera.Camera method*), [12](#)
`toRealDim()` (*BallDetector.BallDetector method*), [14](#)

V

`verify()` (*BallDetector.BallDetector method*), [14](#)
`video_feed()` (*Camera.Camera method*), [12](#)
`VideoCapture`
 module, [14](#)
`VideoCapture` (*class in VideoCapture*), [14](#)
`videoStreaming` (*Camera.Camera attribute*), [12](#)
`vpi_camera_calibration_capture`
 module, [15](#)

W

`w` (*Camera.Camera attribute*), [12](#)
`white_balance()` (*Camera.Camera method*), [12](#)
`worker_uuid` (*Camera.Camera attribute*), [13](#)

Z

`zoomout` (*Camera.Camera attribute*), [13](#)